

Arhitectura calculatoarelor

Curs 3

3. Arhitectura Setului de Instrucțiuni

Cristian Vancea

<https://users.utcluj.ro/~vcristian/AC.html>

Obiective

1. Arhitectura procesorului MIPS
2. Setul de registre
3. Organizarea memoriei
4. Formatul instrucțiunilor
5. Adresarea instrucțiunilor și a datelor
6. Setul de instrucțiuni
7. Tratarea excepțiilor

3. Arhitectura Setului de Instrucțiuni (ISA)

MIPS=**Microprocessor without Interlocked Pipeline Stages**

Definiție: ISA = interfața între hardware și software, la nivelul de jos.

Componente:

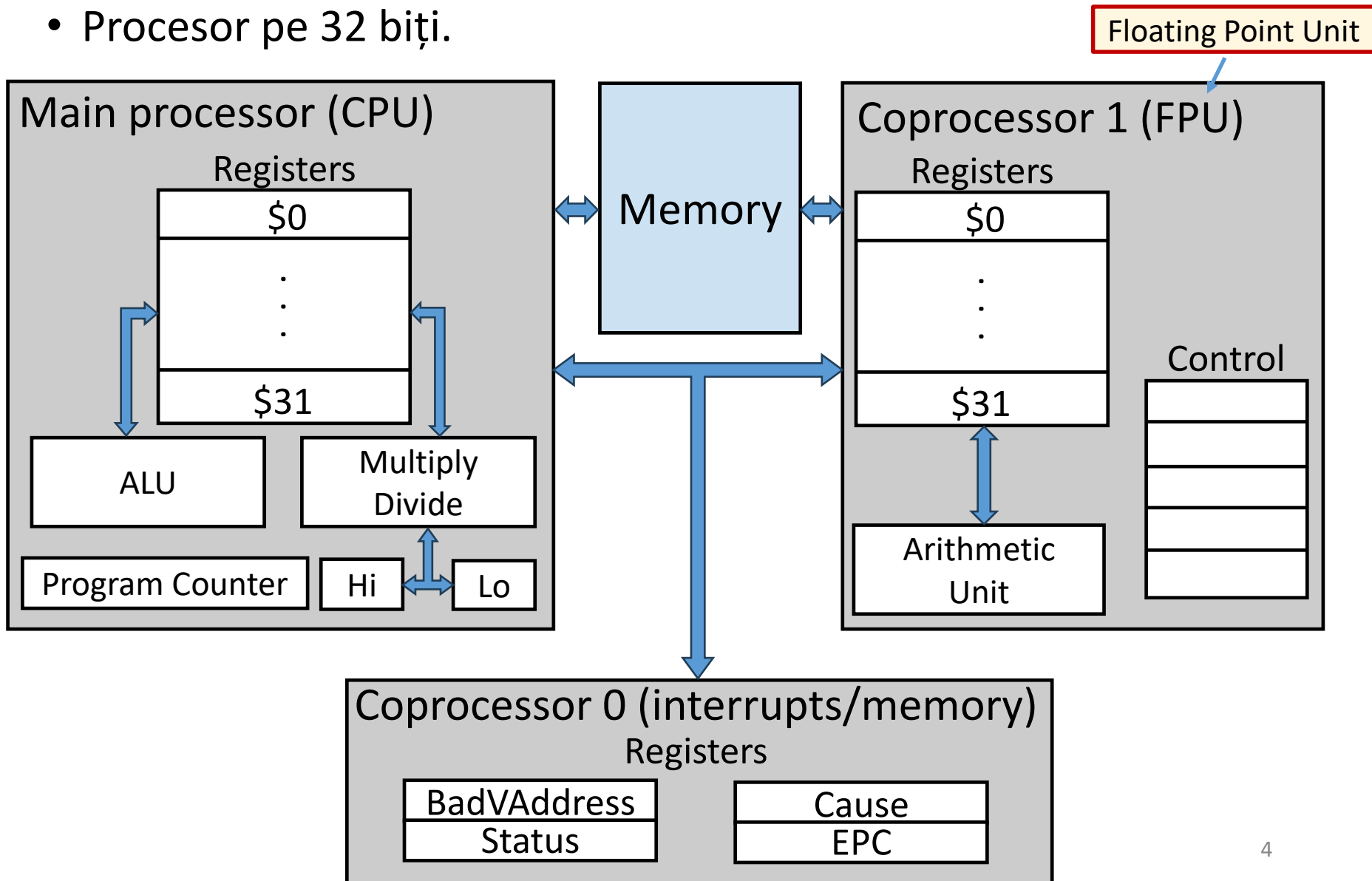
- Setul de registre și organizarea memoriei;
- Tipuri și structuri de date;
- Formatul instrucțiunilor;
- Adresarea instrucțiunilor și a datelor;
- Setul de instrucțiuni;
- Tratarea excepțiilor, protecții, intrare/ieșire.

Fazele de execuție a instrucțiunilor:

- 
1. **Instruction fetch** – citirea din memorie a instrucțiunii;
 2. **Instruction decode** – decodificarea instrucțiunii;
 3. **Operand fetch** – citirea operanzilor;
 4. **Execute** – executarea operațiilor prevăzute de instrucțiune;
 5. **Memory** – accesul la memorie (load sau store);
 6. **Write back** – scrierea rezultatului în registre și actualizarea registrului de adresă a instrucțiunii curente.

3.1 Arhitectura procesorului MIPS

- ISA de tip load-store (cu 3 adrese) cu registre de uz general.
- Procesor pe 32 biți.



3.2 Setul de registre

3.2.1 Main Processor (CPU)

- 32 registre x 32 biți pentru uz general (General Purpose Registers – GPR):
 - R0 are valoarea fixă 0 (orice scriere este ignorată);
 - R31 folosit la apel (*call*) de procedură pentru stocarea adresei de *return*.
- Program Counter (PC) – registru de stocare a adresei instrucțiunii curente.
- Registrele Hi, Lo – stochează rezultate parțiale la înmulțire/împărțire.

Convenții software la registrele GPR		
Nume	Nr. registru	Utilizare
\$zero	0	Permanent 0
\$at	1	Rezervat pentru asamblor
\$v0 - \$v1	2-3	Evaluarea expresiilor și rezultatul funcțiilor
\$a0 - \$a3	4-7	Parametrii
\$t0 - \$t7	8-15	Valori temporare, salvate de apelant
\$s0 - \$s7	16-23	Valori temporare, salvate de funcția apelată
\$t8 - \$t9	24-25	Valori temporare, salvate de apelant
\$k0 - \$k1	26-27	Rezervate pentru nucleul OS (Operating System)
\$gp	28	Pointer global
\$sp	29	Pointer stivă (Top Of Stack)
\$fp	30	Pointer cadru (frame pointer, vârf stivă în apel)
\$ra	31	Adresa de revenire, salvată de instrucțiunea de apel (<i>call</i>)

3.2 Setul de registre

3.2.2 Coprocesoare

Coprocessor 0 (CP0)

- Folosit de sistemul de operare și pentru managementul memoriei.
- Instrucțiuni de scriere/citire a registrelor:
 - **mfc0** \$t0, \$13-> **move from** CP0:Reg13 in CPU:Reg8
 - **mtc0** \$zero, \$12-> **move** CPU:Reg0 **to** CP0:Reg12

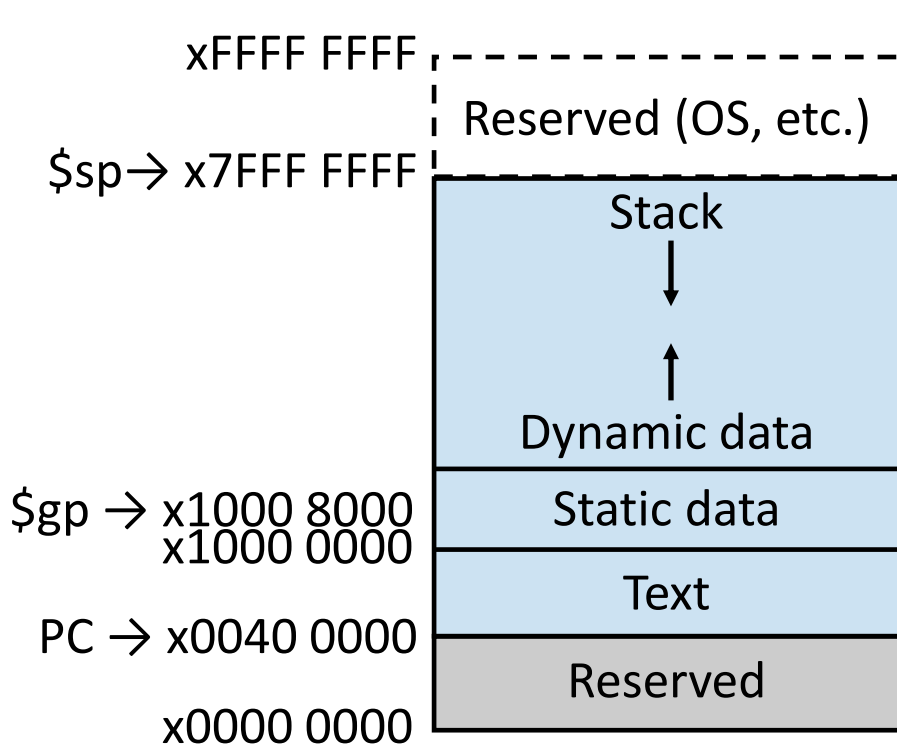
Registreele CP0 cele mai uzuale

Nume	Nr.	Denumire – Utilizare
\$12	12	Stare – stochează starea de funcționare a CPU și biții de întrerupere
\$13	13	Cauză – stochează cauza ultimei întreruperi
\$14	14	EPC (Exception Program Counter) – stochează adresa de reluare a execuției după tratarea unei întreruperi
\$8	8	BadVAddr – stochează ultima adresa virtuală care a provocat o eroare de acces la memorie

Coprocessor 1 (CP1)

- folosit pentru calcule în virgulă mobilă;
- are 32 registre x 32 biți în virgulă mobilă (f0-f31) – FPR (Floating Point Registers);
- are 5 registre de control.

3.3 Organizarea memoriei



Proprietăți

- Adrese pe 32 biți => dimensiunea= 2^{32} octeți.
- Adresare la octet.
- 1 cuvânt de memorie = 32 biți => 2^{30} cuvinte.
- Aliniere în memorie:
 - Big/little-endian la alegere;
 - Pentru cuvinte de 32 biți => adresa multiplu de 4;
 - Pentru semicuvinte de 16 biți => adresa multiplu de 2.

Tipuri de date

- întregi, 32 biți;
- reale, 32 biți - float;
- reale, 64 biți - double.

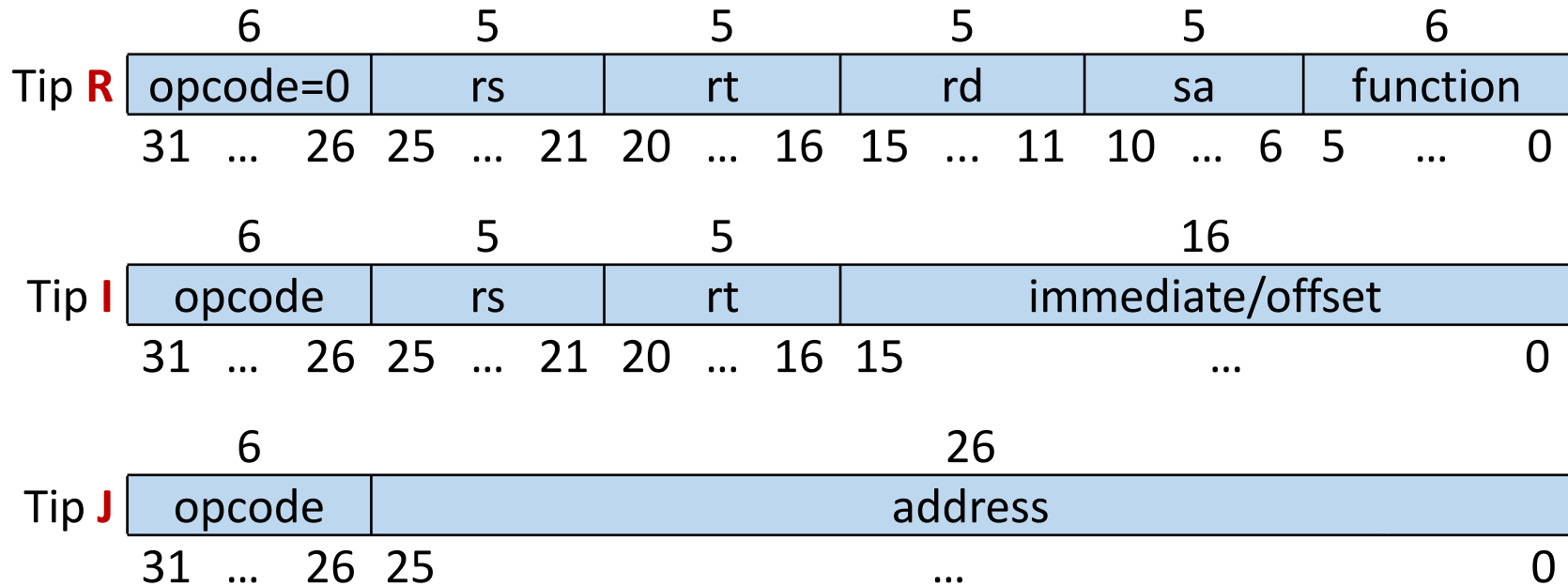
Încărcare date în GPR

- cuvinte, 32 biți;
- semicuvinte, 16 biți;
- octeți, 8 biți.

Încărcare date în FPR

- Numere reale pe 32 biți;
- Numere reale pe 64 biți, numai în registrele cu indice par (f0, f2, etc.)

3.4 Formatul instrucțiunilor



Se folosesc 3 tipuri de instrucțiuni pe 32 de biți:

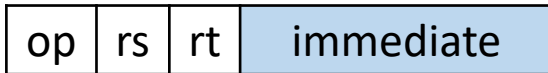
- **Tip R** (Register) – pentru operații aritmetice și logice; au câmpul *opcode*=0.
- **Tip I** (Immediate) – pentru operații aritmetice și logice cu valoare imediată, pentru load/store cu memoria și pentru salturi condiționate (branch).
- **Tip J** (Jump) – pentru salturi necondiționate (jump).

3.4 Formatul instrucțiunilor

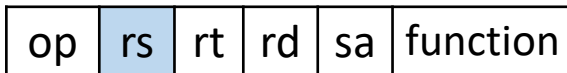
Câmp	Descriere
opcode	codul instrucțiunii (este 0 pentru instrucțiuni de tip R) → 6 biți
rs	indexul primului registru sursă → 5 biți
rt	indexul celui de al 2-lea registru sursă (la tip R) / destinație (la tip I) → 5 biți
rd	indexul registrului destinație (la tip R) → 5 biți
sa	cu câți biți se face deplasarea (shift amount) → 5 biți
function	codul unic pentru instrucțiuni de tip R, deoarece toate au opcode=0 → 6 biți
immediate / offset	valoare pentru operații logice și aritmetice cu o constantă (imediat) → 16 biți
	deplasamentul (offset) în octeți, pentru operațiile load/store cu memoria → 16 biți
	deplasamentul (offset) în instrucțiuni, relativ la PC+4, pentru salturi condiționate → 16 biți
address	indexul în instrucțiuni, la salturi necondiționate; se deplasează cu 2 biți la stânga și conferă cei 28 de biți mai puțin semnificativi ai adresei de salt → 26 biți

3.5 Adresarea instrucțiunilor și a datelor

Adresare cu imediat



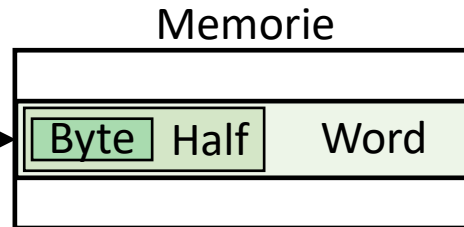
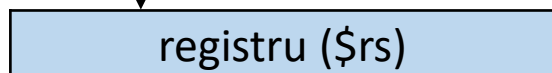
Adresare cu registru



Blocul de registre



Adresare cu bază și index

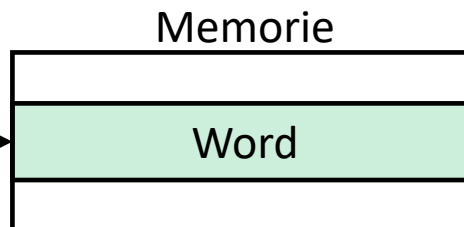
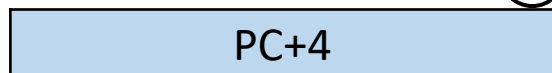
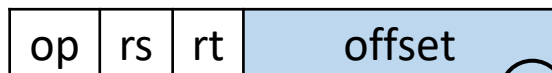


Indexat $\rightarrow \$rs + \text{offset}$

Registru indirect $\rightarrow \$rs + 0$

Adresare absolută $\rightarrow \$0 + \text{offset}$

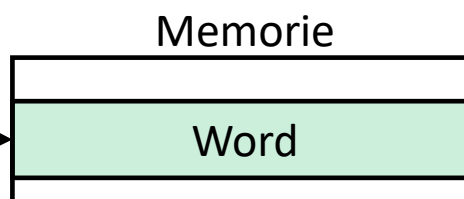
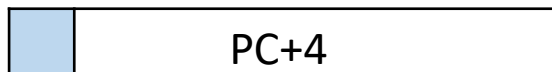
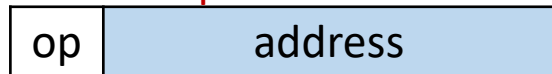
Adresare relativă la PC+4



La salt condiționat (branch):

Adresa salt $\leftarrow (PC+4) + (\text{offset} \times 4)$

Adresare pseudo-directă



La salt necondiționat (jump):

Adresa salt $\leftarrow (PC+4)[31:28] || \text{address}[25:0] || 00$

3.6 Setul de instrucțiuni

3.6.1 Categorii

Instrucțiune	Exemplu	Efect	Descriere
Aritmetice			
add	add \$s1, \$s2, \$s3	$\$s1 \leftarrow \$s2 + \$s3$	adunare cu 3 operanzi
subtract	sub \$s1, \$s2, \$s3	$\$s1 \leftarrow \$s2 - \$s3$	scădere cu 3 operanzi
add immediate	addi \$s1, \$s2, 100	$\$s1 \leftarrow \$s2 + 100$	adunare cu o constantă
Logice			
and	and \$s1, \$s2, \$s3	$\$s1 \leftarrow \$s2 \& \$s3$	and pe biți cu 3 operanzi
or	or \$s1, \$s2, \$s3	$\$s1 \leftarrow \$s2 \mid \$s3$	or pe biți cu 3 operanzi
nor	nor \$s1, \$s2, \$s3	$\$s1 \leftarrow \overline{\$s2 \mid \$s3}$	nor pe biți cu 3 operanzi
and immediate	andi \$s1, \$s2, 100	$\$s1 \leftarrow \$s2 \& 100$	and pe biți cu o constantă
or immediate	ori \$s1, \$s2, 100	$\$s1 \leftarrow \$s2 \mid 100$	or pe biți cu o constantă
shift left logical	sll \$s1, \$s2, 10	$\$s1 \leftarrow \$s2 \ll 10$	deplasare la stânga cu o constantă
shift right logical	srl \$s1, \$s2, 10	$\$s1 \leftarrow \$s2 \gg 10$	deplasare la dreapta cu o constantă

3.6 Setul de instrucțiuni

3.6.2 Categori (cont)

Instrucțiune	Exemplu	Efect	Descriere
Transfer de date			
load word	lw \$s1, 100(\$s2)	$\$s1 \leftarrow \text{Mem}[\$s2+100]$	citire word din memorie în registru
store word	sw \$s1, 200(\$s2)	$\text{Mem}[\$s2+200] \leftarrow \$s1$	scriere word din registru în memorie
load half unsigned	lhu \$s1, 104(\$s2)	$\$s1 \leftarrow \text{Mem}[\$s2+104]$	citire half word din memorie în registru
store half	sh \$s1, 112(\$s2)	$\text{Mem}[\$s2+112] \leftarrow \$s1$	scriere half word din registru în memorie
load byte unsigned	lbu \$s1, 106(\$s2)	$\$s1 \leftarrow \text{Mem}[\$s2+106]$	citire byte din memorie în registru
store byte	sb \$s1, 106(\$s2)	$\text{Mem}[\$s2+106] \leftarrow \$s1$	scriere byte din registru în memorie
load upper Immediate	lui \$s1, 98	$\$s1 \leftarrow 98 \times 2^{16}$	încărcarea valorii 98 în jumătatea superioară a registrului
Salt necondiționat			
jump	j 16	$\text{PC} \leftarrow (\text{PC}+4)_{31:28} \parallel \text{x0000040}$	salt la adresă
jump register	jr \$ra	$\text{PC} \leftarrow \$ra$	revenire din procedură (return)
jump and link	jal 16	$\$ra \leftarrow \text{PC}+4;$ $\text{PC} \leftarrow (\text{PC}+4)_{31:28} \parallel \text{x0000040}$	apel de procedură cu salvarea adresei de revenire (return)

3.6 Setul de instrucțiuni

3.6.2 Categori (cont)

Instrucțiune	Exemplu	Efect	Descriere
Salt condiționat			
branch on equal	beq \$s1, \$s2, 25	$(\$s1 = \$s2) \rightarrow PC \leftarrow (PC+4) + 100$	salt relativ la PC+4 în caz de egalitate
branch on not equal	bne \$s1, \$s2, 25	$(\$s1 \neq \$s2) \rightarrow PC \leftarrow (PC+4) + 100$	salt relativ la PC+4 în caz de inegalitate
Atribuii condiționate			
set on less than	slt \$s1, \$s2, \$s3	$(\$s2 < \$s3) \rightarrow \$s1 \leftarrow 1$ else $\$s1 \leftarrow 0$	atribuire condiționată de comparație cu semn
set on less than immediate	slti \$s1, \$s2, 100	$(\$s2 < 100) \rightarrow \$s1 \leftarrow 1$ else $\$s1 \leftarrow 0$	atribuire condiționată de comparație cu semn, cu o constantă
set on less than unsigned	sltu \$s1, \$s2, \$s3	$(\$s2 < \$s3) \rightarrow \$s1 \leftarrow 1$ else $\$s1 \leftarrow 0$	atribuire condiționată de comparație fără semn
set on less than immediate unsigned	sltiu \$s1, \$s2, 100	$(\$s2 < 100) \rightarrow \$s1 \leftarrow 1$ else $\$s1 \leftarrow 0$	atribuire condiționată de comparație fără semn, cu o constantă

3.6 Setul de instrucțiuni

3.6.3 Selecție

Assembly	Type	RTL Abstract	Instruction
add \$rd, \$rs, \$rt	R	$RF[rd] \leftarrow RF[rs] + RF[rt]$	add
sub \$rd, \$rs, \$rt	R	$RF[rd] \leftarrow RF[rs] - RF[rt]$	subtract
addi \$rt, \$rs, imm	I	$RF[rt] \leftarrow RF[rs] + S_Ext(imm)$	add immediate
addu \$rd, \$rs, \$rt	R	$RF[rd] \leftarrow RF[rs] + RF[rt]$	add unsigned
subu \$rd, \$rs, \$rt	R	$RF[rd] \leftarrow RF[rs] - RF[rt]$	subtract unsigned
mult \$rs, \$rt	R	$Hi, Lo \leftarrow RF[rs] \times RF[rt]$	multiplication
multu \$rs, \$rt	R	$Hi, Lo \leftarrow RF[rs] \times RF[rt]$	unsigned multiplication
div \$rs, \$rt	R	$Lo \leftarrow RF[rs] / RF[rt]$ $Hi \leftarrow RF[rs] \bmod RF[rt]$	division
divu \$rs, \$rt	R	$Lo \leftarrow RF[rs] / RF[rt]$ $Hi \leftarrow RF[rs] \bmod RF[rt]$	unsigned division
mfhi \$rd	R	$RF[rd] \leftarrow Hi$	move from Hi
mflo \$rd	R	$RF[rd] \leftarrow Lo$	move from Lo

3.6 Setul de instrucțiuni

3.6.3 Selecție (cont)

Assembly	Type	RTL Abstract	Instruction
lw \$rt, offset(\$rs)	I	$RF[rt] \leftarrow M[RF[rs] + S_Ext(offset)]$	load word
sw \$rt, offset(\$rs)	I	$M[RF[rs] + S_Ext(offset)] \leftarrow RF[rt]$	store word
lb \$rt, offset(\$rs)	I	$RF[rt] \leftarrow S_Ext(M[RF[rs] + S_Ext(offset)])$	load byte
sb \$rt, offset(\$rs)	I	$M[RF[rs] + S_Ext(offset)] \leftarrow \text{byte}(RF[rt])$	store byte
lui \$rt, imm	I	$RF[rt] \leftarrow \text{imm} \mid \mid x0000$	load upper immediate
beq \$rs, \$rt, offset	I	if $RF[rs]=RF[rt]$ then $PC \leftarrow (PC+4) + S_Ext(offset) \ll 2$	branch on equal
bne \$rs, \$rt, offset	I	if $RF[rs] \neq RF[rt]$ then $PC \leftarrow (PC+4) + S_Ext(offset) \ll 2$	branch on not equal
bgez \$rs, offset	I	if $RF[rs] \geq 0$ then $PC \leftarrow (PC+4) + S_Ext(offset) \ll 2$	branch on greater or equal to zero
bltz \$rs, offset	I	if $RF[rs] < 0$ then $PC \leftarrow (PC+4) + S_Ext(offset) \ll 2$	branch on less than zero

3.6 Setul de instrucțiuni

3.6.3 Selecție (cont)

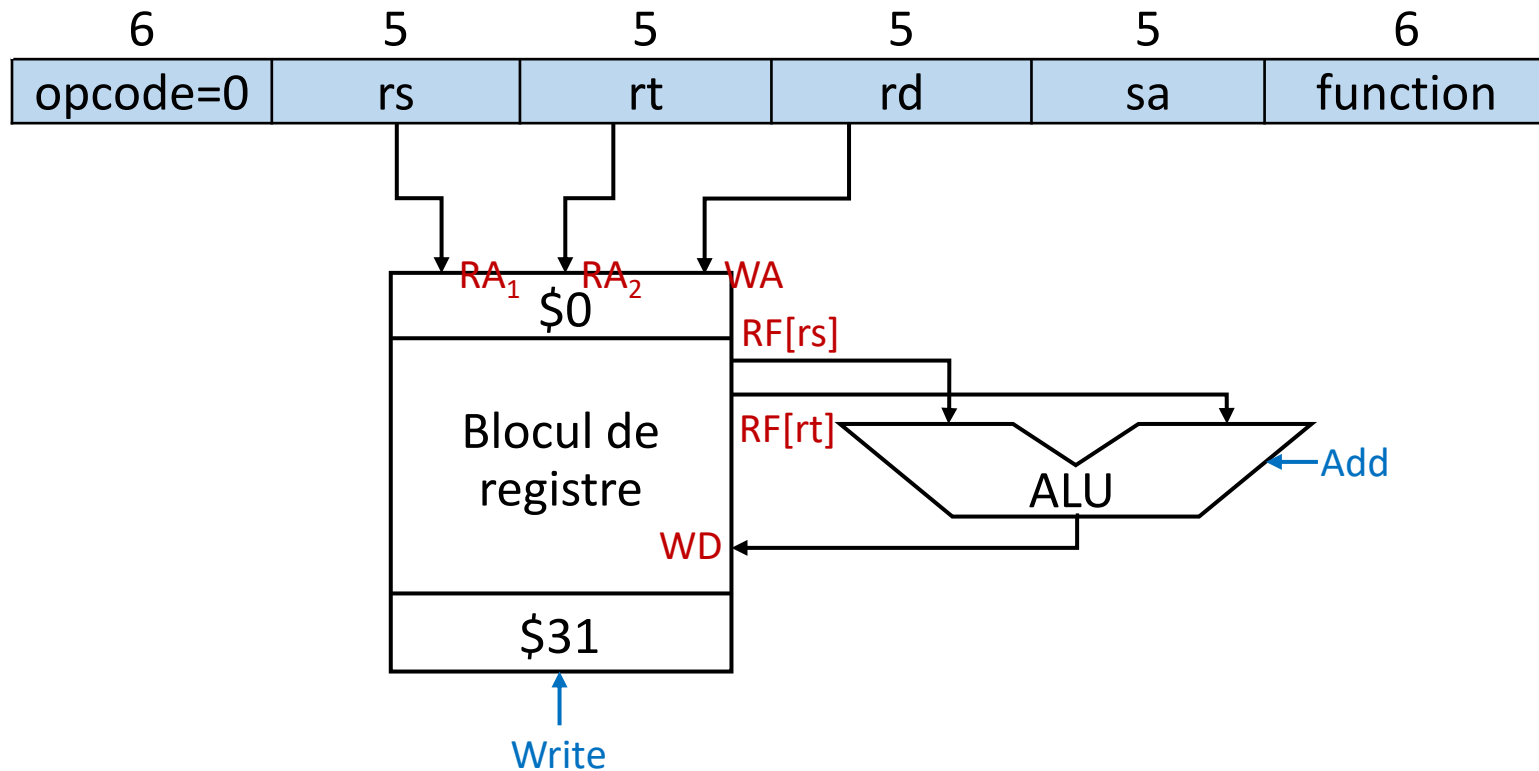
Assembly	Type	RTL Abstract	Instruction
slt \$rd, \$rs, \$rt	R	if $RF[rs] < RF[rt]$ then $RF[rd] \leftarrow 1$ else $RF[rd] \leftarrow 0$	set on less than
slti \$rt, \$rs, imm	I	if $RF[rs] < imm$ then $RF[rt] \leftarrow 1$ else $RF[rt] \leftarrow 0$	set on less than immediate
sltu \$rd, \$rs, \$rt	R	if $RF[rs] < RF[rt]$ then $RF[rd] \leftarrow 1$ else $RF[rd] \leftarrow 0$	set on less than unsigned
sltiu \$rt, \$rs, imm	I	if $RF[rs] < imm$ then $RF[rt] \leftarrow 1$ else $RF[rt] \leftarrow 0$	set on less than immediate unsigned
j address	J	$PC \leftarrow (PC+4)[31:28] \parallel address \parallel 00$	jump
jal address	J	$RF[31] \leftarrow PC+4$ $PC \leftarrow (PC+4)[31:28] \parallel address \parallel 00$	jump and link
jr \$rs	R	$PC \leftarrow RF[rs]$	jump register

3.6 Setul de instrucțiuni

3.6.4 Diagrama de procesare

- Instrucțiuni aritmetice-logice, de tip R

add \$rd, \$rs, \$rt $RF[rd] \leftarrow RF[rs] + RF[rt]$



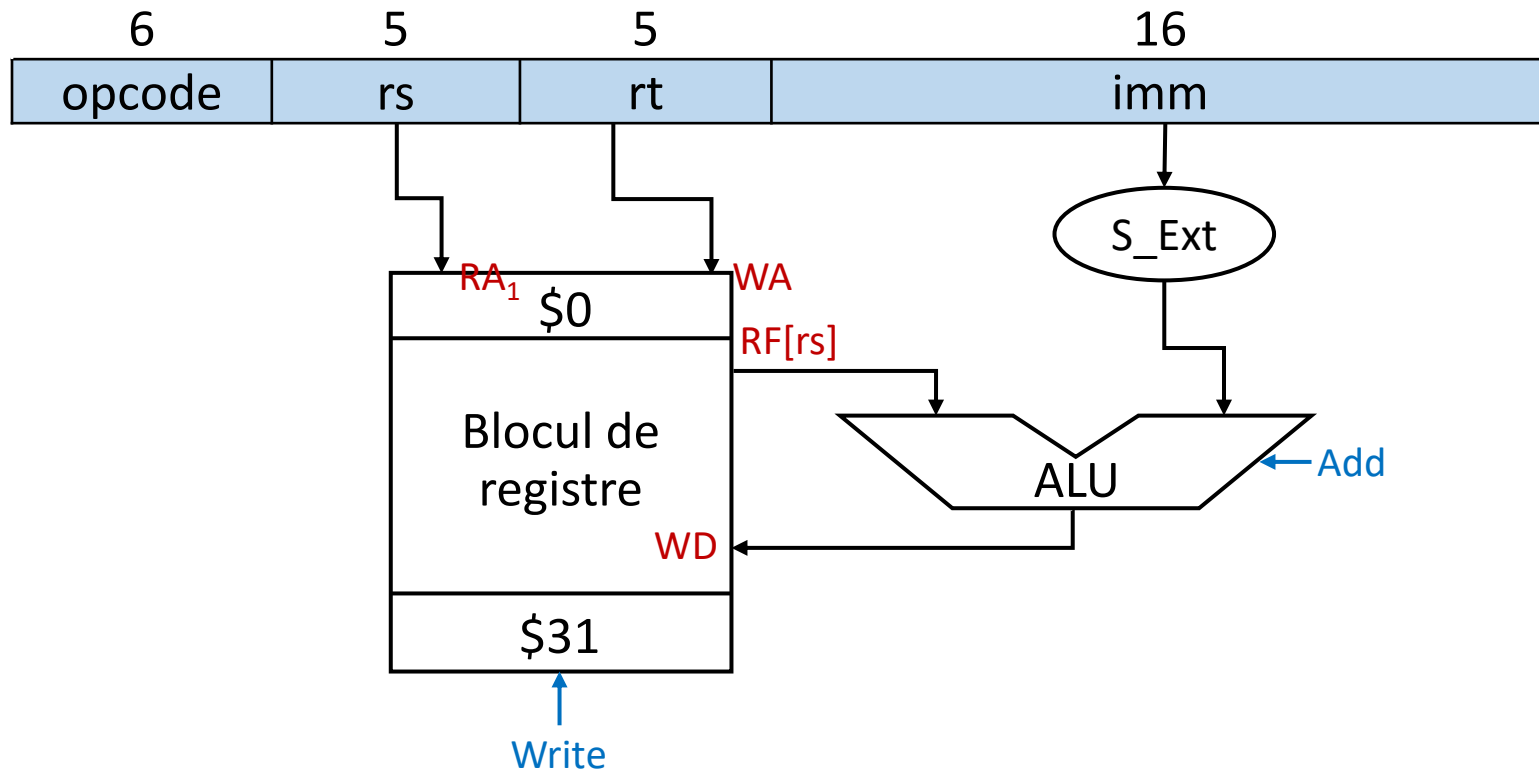
Blocul de registre = memorie 32x32 cu 2 porturi de citire (deci 2 x read address: RA₁ și RA₂) și 1 port de scriere (WA – write address, WD – write data). Poate oferi la ieșire valorile a 2 registre și poate scrie într-un al 3-lea.

3.6 Setul de instrucțiuni

3.6.4 Diagrama de procesare (cont)

- Instrucțiuni aritmetice cu imediat, de tip I

addi \$rt, \$rs, imm $RF[rt] \leftarrow RF[rs] + S_Ext(imm)$

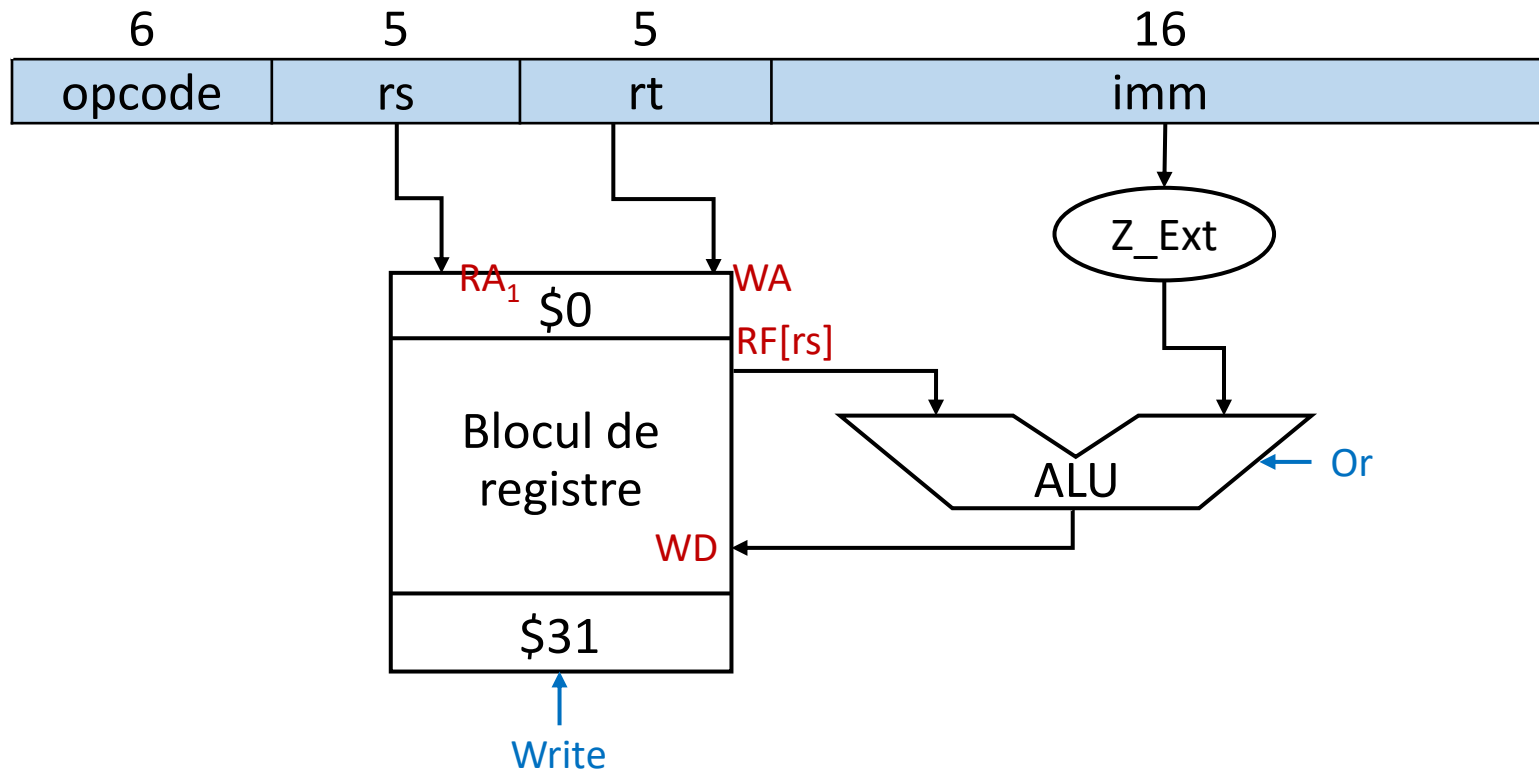


3.6 Setul de instrucțiuni

3.6.4 Diagrama de procesare (cont)

- Instrucțiuni logice cu imediat, de tip I

ori \$rt, \$rs, imm $RF[rt] \leftarrow RF[rs] \mid Z_Ext(imm)$

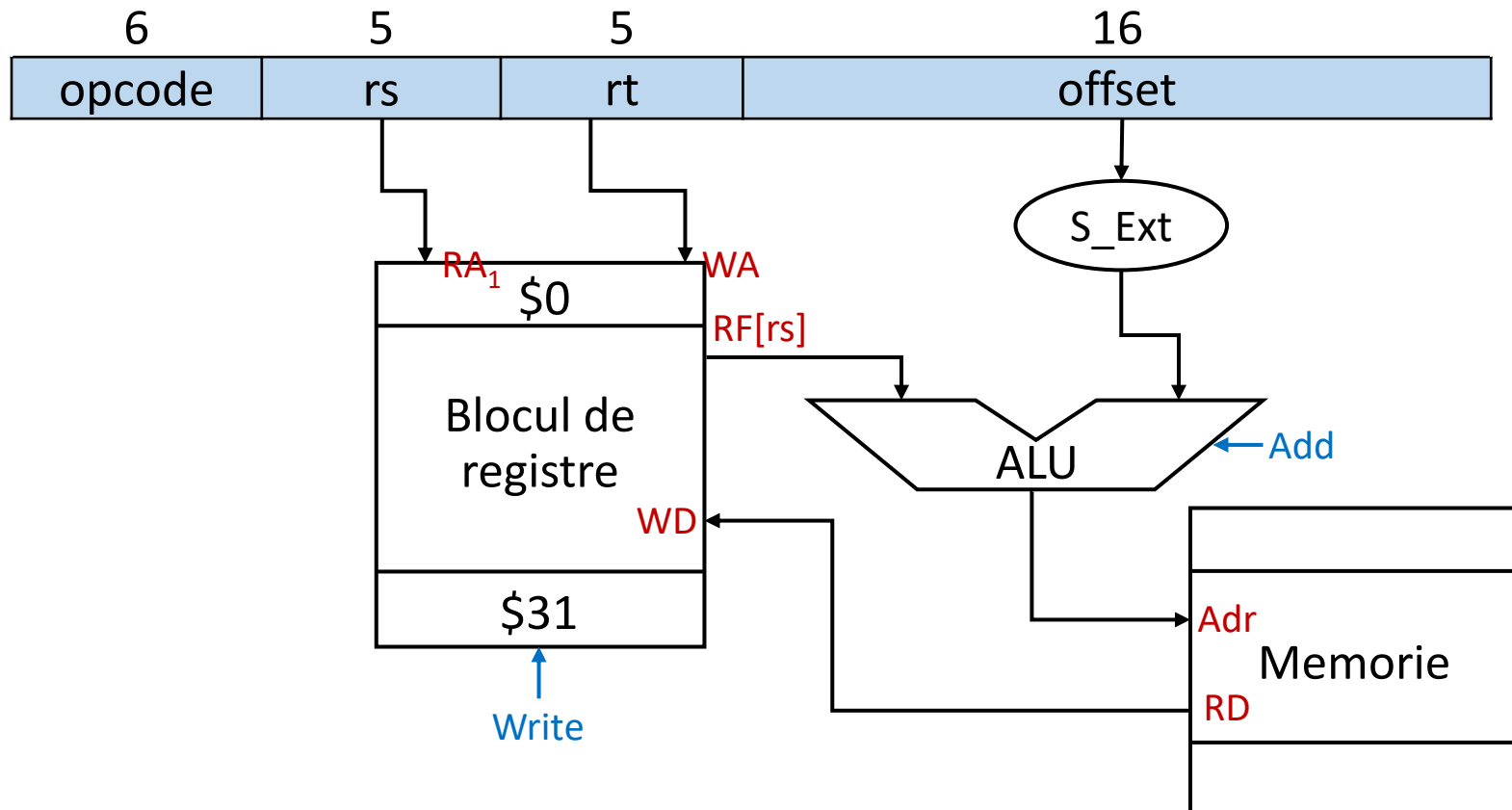


3.6 Setul de instrucțiuni

3.6.4 Diagrama de procesare (cont)

- Instrucțiunea Load Word, de tip I

lw \$rt, offset(\$rs) $RF[rt] \leftarrow M[RF[rs] + S_Ext(offset)]$

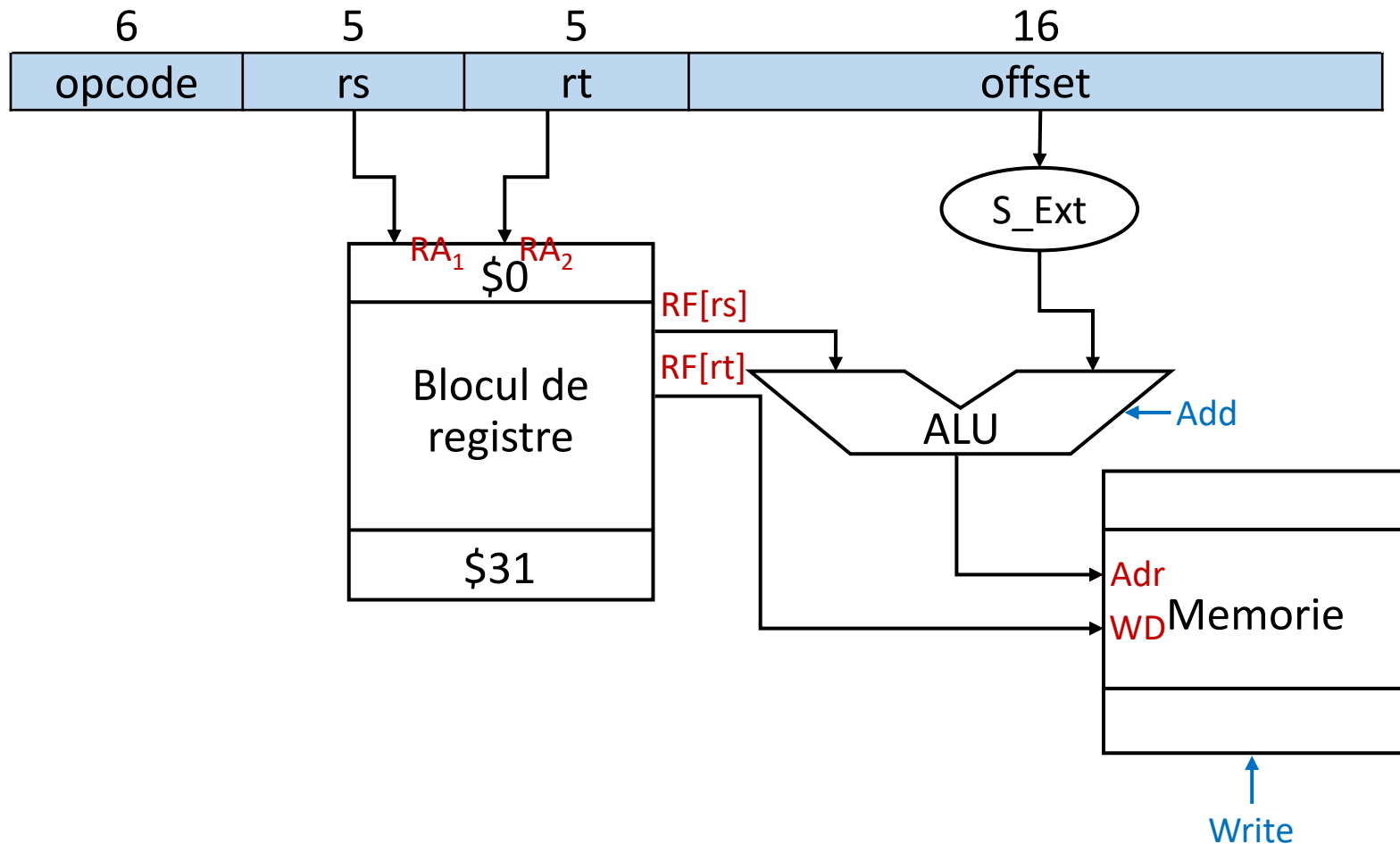


3.6 Setul de instrucțiuni

3.6.4 Diagrama de procesare (cont)

- Instrucțiunea Store Word, de tip I

sw \$rt, offset(\$rs) $M[RF[rs] + S_Ext(offset)] \leftarrow RF[rt]$

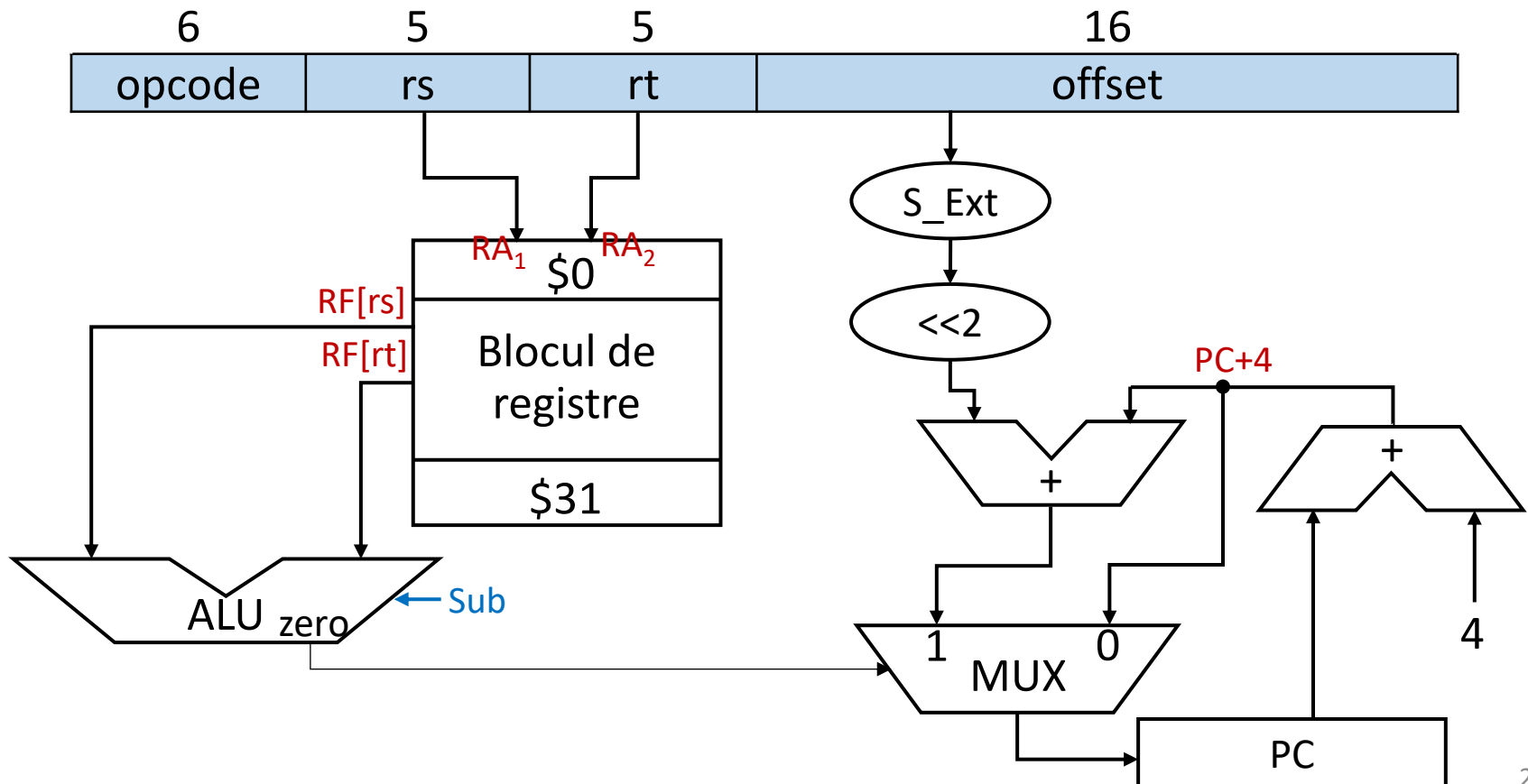


3.6 Setul de instrucțiuni

3.6.4 Diagrama de procesare (cont)

- Instrucțiunea Branch on Equal, de tip I

beq \$rs, \$rt, offset if $RF[rs]=RF[rt]$ then $PC \leftarrow (PC+4) + S_Ext(offset) \ll 2$
else $PC \leftarrow PC+4$

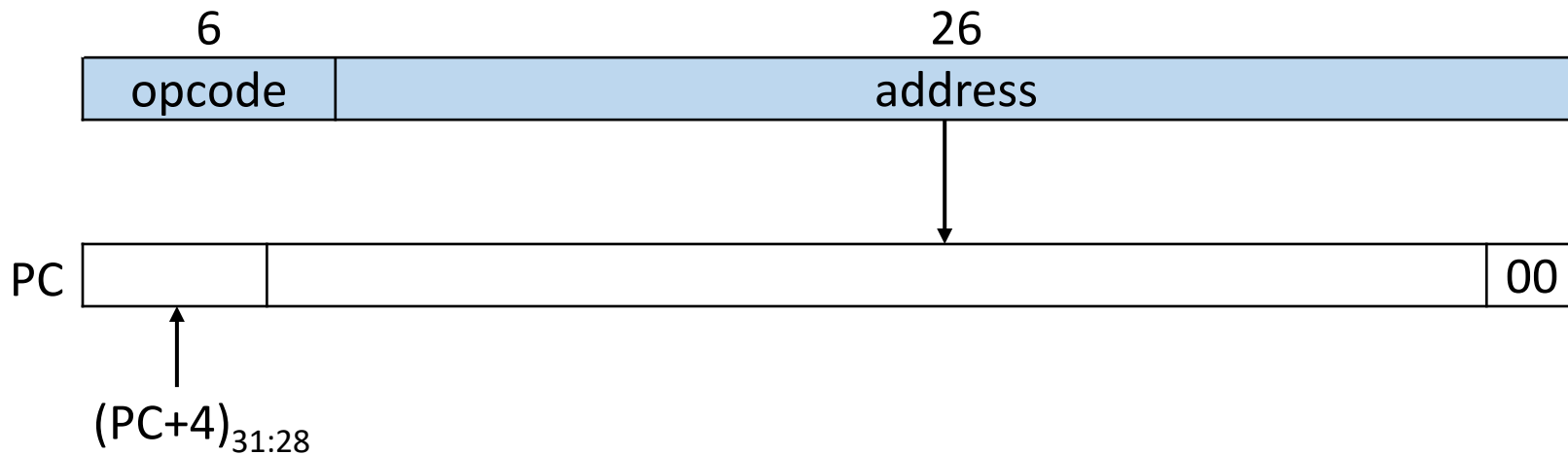


3.6 Setul de instrucțiuni

3.6.4 Diagrama de procesare (cont)

- Instrucțiunea Jump, de tip J

j address $PC \leftarrow (PC+4)[31:28] \parallel \text{address} \parallel 00$



3.6 Setul de instrucțiuni

3.6.5 Pseudo-instrucțiuni

- Nu sunt suportate de processor.
- Sunt suportate de compiler.
- Sunt implementate cu instrucțiuni ale procesorului (native).
- Sunt utile pentru simplificarea în programare.

Exemple

ABSolute value

abs \$rd, \$rs

Implementare 1

addu \$rd, \$zero, \$rs

bgez \$rs, 1

sub \$rd, \$zero, \$rs

Implementare 2

slt \$ra, \$rs, \$zero

bne \$ra, \$zero, NEG

add \$rd, \$rs, \$zero

j DONE

NEG: sub \$rs, \$zero, \$rs

DONE:

Branch on EQual to Zero

beqz \$rs, label

beq \$rs, \$zero, label

3.6 Setul de instrucțiuni

3.6.5 Pseudo-instrucțiuni (cont)

Branch on Greater or Equal bge \$rs, \$rt, label	Implementare slt \$at, \$rs, \$rt beq \$at, \$zero, label	MOVE move \$rd, \$rs	Implementare addu \$rd, \$zero, \$rs
Branch on Greater or Equal Unsigned bgeu \$rs, \$rt, label	Implementare sltu \$at, \$rs, \$rt beq \$at, \$zero, label	NEGate neg \$rd, \$rs	Implementare sub \$rd, \$zero, \$rs
Branch on Greater Than bgt \$rs, \$rt, label	Implementare slt \$at, \$rt, \$rs bne \$at, \$zero, label	NO OPeration noop noop	Implementare or \$zero,\$zero,\$zero sll \$zero, \$zero, 0
Load Immediate li \$rt, value	Implementare ori \$rt, \$zero, value		

3.7 Tratarea excepțiilor

Se realizează în 3 pași:

1. **Salvare PC.** Registrul **CP0:EPC** (Exception Program Counter) memorează:
 - Adresa instrucțiunii care generează excepția.
2. **Salt la rutina de tratare a întreruperilor și se salvează cauza:**
 - $PC \leftarrow x80000180$;
 - Registrul **CP0:Cause** se scrie cu codul asociat cauzei întreruperii:
 - Întrerupere hardware = **0**;
 - Adresă memorie ilegală (la *load/fetch* sau *store*) = **4 sau 5**;
 - Eroare magistrală (la *fetch* sau *load/store*) = **6 sau 7**;
 - Instrucțiune *syscall* = **8**;
 - Cod operație ilegal, rezervat sau nedefinit = **10**;
 - Overflow la operații pe întreg = **12**;
 - Întrerupere software (trap) = **13**;
 - Orice excepție la calcule cu virgulă mobilă = **15**.
3. **CPU intră în modul kernel** (cu privilegii de acces la zone de memorie și CP0):
 - În registrul **CP0:Status** biții *CPU mode* $\leftarrow 00$ (kernel mode).

Probleme propuse

1. Care este spațiul de adresare pentru procesorul MIPS (memoria care poate fi accesată)?
2. Folosind instrucțiuni MIPS native, scrieți pseudo-instrucțiuni pentru următoarele operații:
a) valoare absolută (modul); b) Branch on Greater Than; c) Branch on Less or Equal; 2) Swap two registers (schimbă două registre).
3. Definiți instrucțiuni noi pentru:
a) LWR (load word register) – însumează două registre pentru calculul adresei de memorie de unde să citească;
b) SWR (store word register) – idem, pentru stocare în memorie;
c) LWA, SWA – load/store cu un singur registru pentru calculul adresei.

Pași: stabiliți formatul de instrucțiune optim, RTL abstract și desenați diagrama de procesare.

Referințe

- [1] F. Oniga, “De la bit la procesor. Introducere în arhitectura calculatoarelor”, Cap. 7.4, 9.1, 9.2, Ed. U.T. Press, 2019, ISBN 978-606-737-366-0.
- [2] D. A. Patterson, J. L. Hennessy, “Computer Organization and Design: The Hardware/Software Interface”, 5th edition, Cap. 2.2, 2.10, Anexa A5, A7, Ed. Morgan–Kaufmann, 2013.
- [3] J. L. Hennessy, D. A. Patterson, “Computer Organization and Design: A Quantitative Approach”, 5th edition, Anexa A9, Ed. Morgan–Kaufmann, 2011.
- [4] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers, Volume I: Introduction to the MIPS32™ Architecture”.
- [5] MIPS Technologies, Inc., “MIPS32™ Architecture for Programmers Volume II: The MIPS32™ Instruction Set”.